Matrix Benchmark Timing Report

Codex benchmark sweep for the Matrix project

March 15, 2026

1 Benchmark Setup

This report summarizes a quick but broad timing sweep for the four shipped test programs: matrix multiplication, REF, RREF, and LU. The benchmark data was generated by `benchmarks/scripts/run_benchmark` and written to `benchmarks/data/timings.csv` and `benchmarks/data/summary.csv`.

- Matrix sizes: 250, 500, 1000, and 1500
- MPI process counts: 1, 2, 4, 8, and 16
- Machine: 32 logical CPU cores, about 24 GiB of available RAM at run time
- Timing source: kernel timings printed by the test executables themselves

The goal of this sweep was to stay fast enough to rerun easily while still capturing how scaling changes across several problem sizes. These timings are not intended to represent the maximum matrix size now supported by the heap backed matrix implementation.

2 Best Timing Summary

Operation	N	Seq. (s)	Best MPI np	Best MPI (s)	Speedup
LU	250	0.005	4	0.005	1.00
LU	500	0.045	2	0.040	1.13
LU	1000	0.332	1	0.327	1.02
LU	1500	1.187	8	1.123	1.06
Multiply	250	0.017	8	0.004	4.25
Multiply	500	0.141	8	0.037	3.81
Multiply	1000	1.341	8	0.385	3.48
Multiply	1500	5.153	16	1.391	3.70
REF	250	0.006	4	0.003	2.00
REF	500	0.057	8	0.016	3.56
REF	1000	0.440	8	0.063	6.98
REF	1500	1.670	8	0.206	8.11
RREF	250	0.025	2	0.003	8.33
RREF	500	0.097	8	0.014	6.93
RREF	1000	0.781	8	0.087	8.98
RREF	1500	3.058	8	0.208	14.70

Table 1: Best observed MPI kernel time at each benchmarked matrix size.

3 Observations

- Matrix multiplication scaled well through `np=8` and remained strong at `np=16`. The best observed result was 1.391 seconds at $N = 1500$ with `np=16`, compared with 5.153 seconds sequential.
- REF showed its best results at `np=8` for the three larger sizes. At $N = 1500$, the best MPI run was 0.206 seconds versus 1.670 seconds sequential, which is about an 8.11× speedup.
- RREF produced the strongest scaling in this sweep. At $N = 1500$, the best MPI timing was 0.208 seconds at `np=8`, compared with 3.058 seconds sequential, a speedup of about 14.70×.
- LU remained effectively sequential in practice. The current test harness only performs the decomposition on rank 0, so additional MPI processes mostly add launch and synchronization overhead rather than real distributed compute.
- The raw wall times in `timings.csv` are much larger than the kernel times for small matrices because `mpirun` startup dominates those runs. The kernel timings are the correct numbers to use when comparing the algorithm implementations inside the existing test drivers.

4 Figures

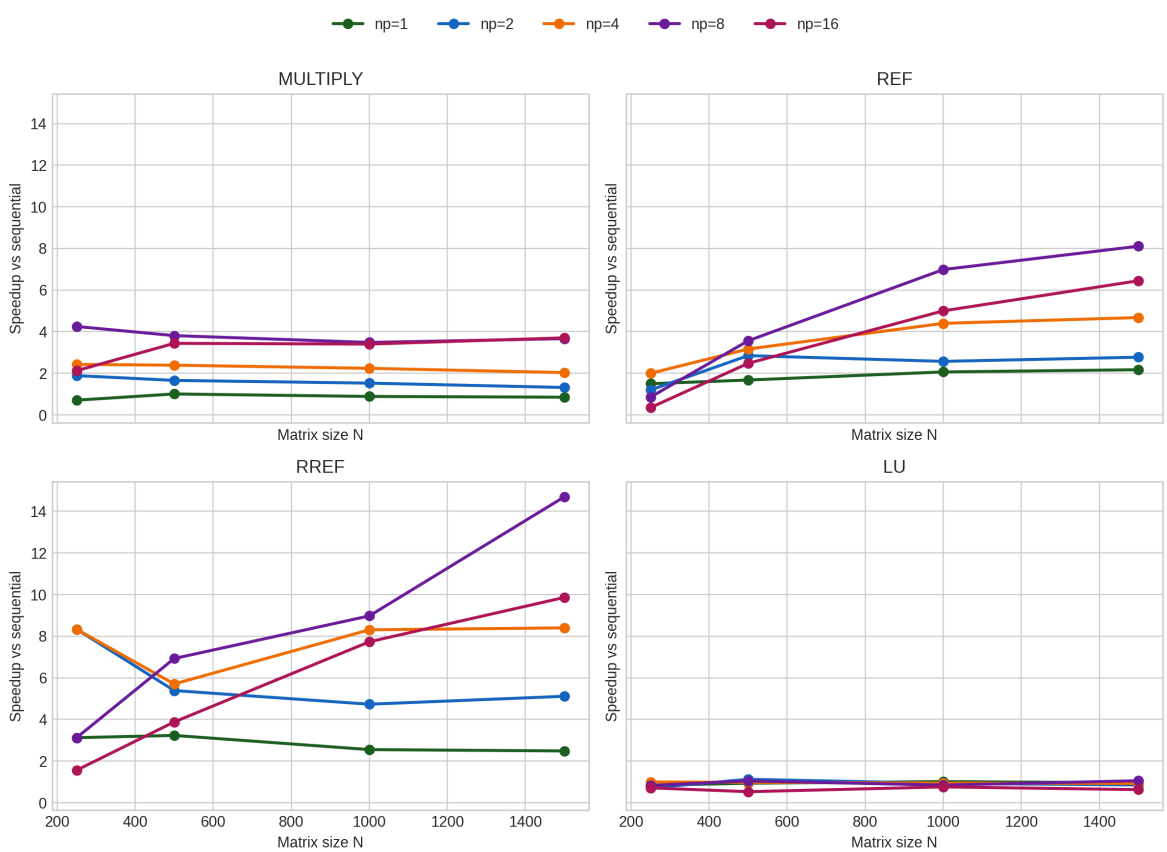

Figure 1: Speedup relative to the sequential baseline across all four operations.

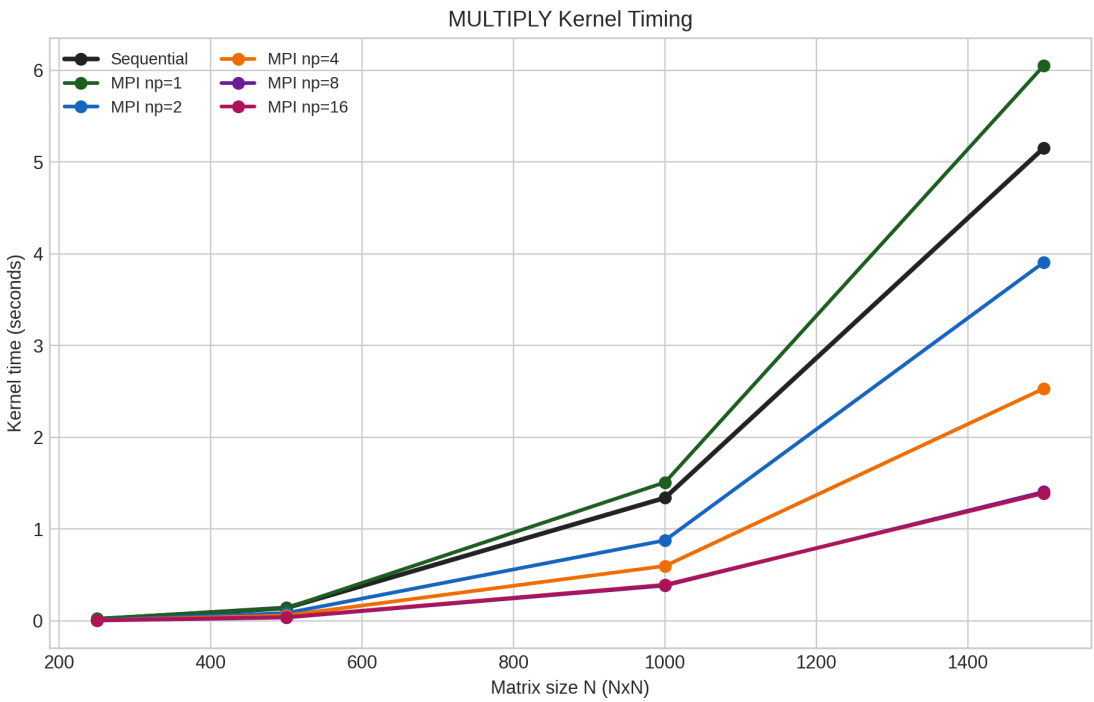

Figure 2: Matrix multiplication kernel timings across process counts.

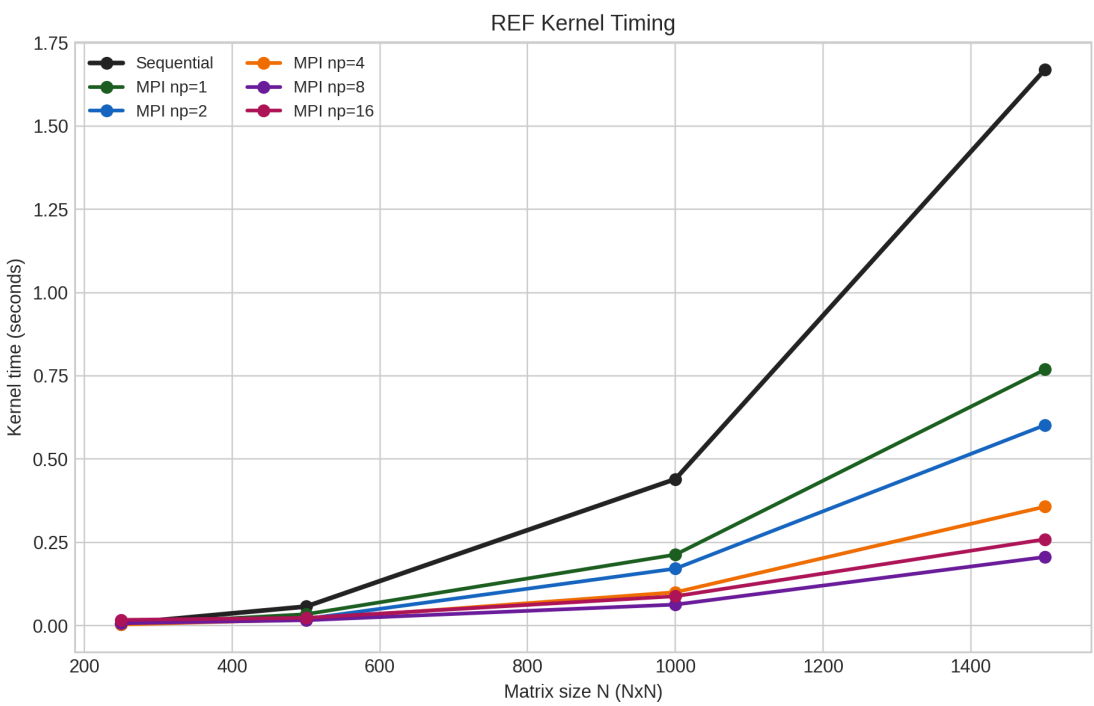

Figure 3: REF kernel timings across process counts.

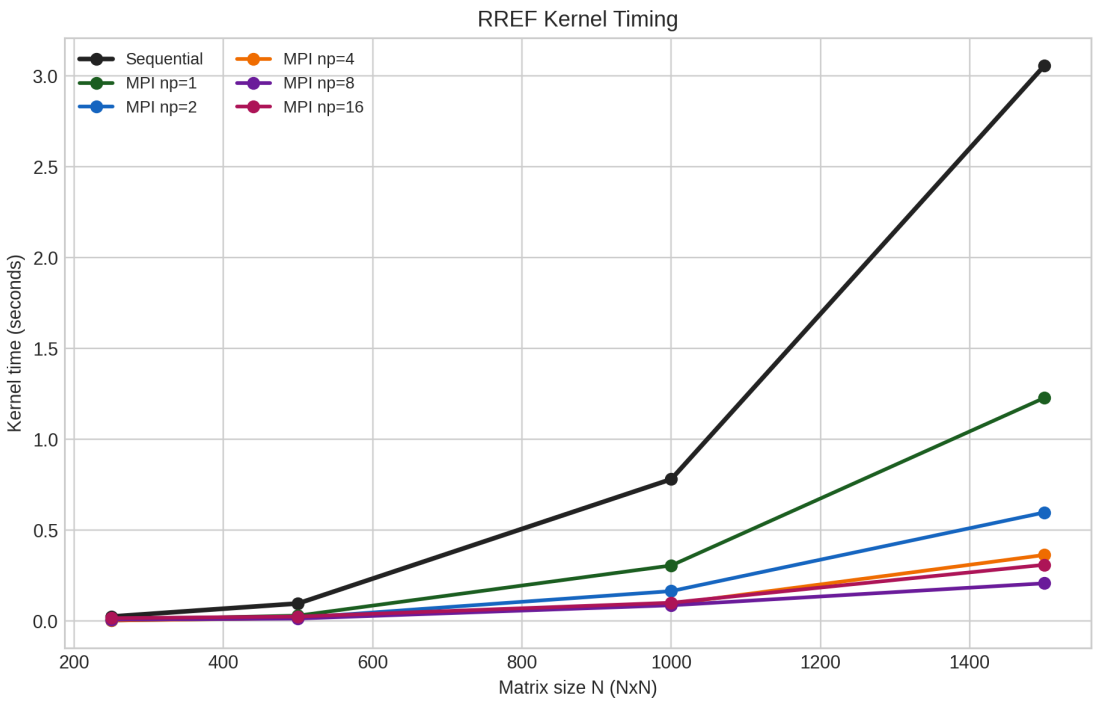

Figure 4: RREF kernel timings across process counts.

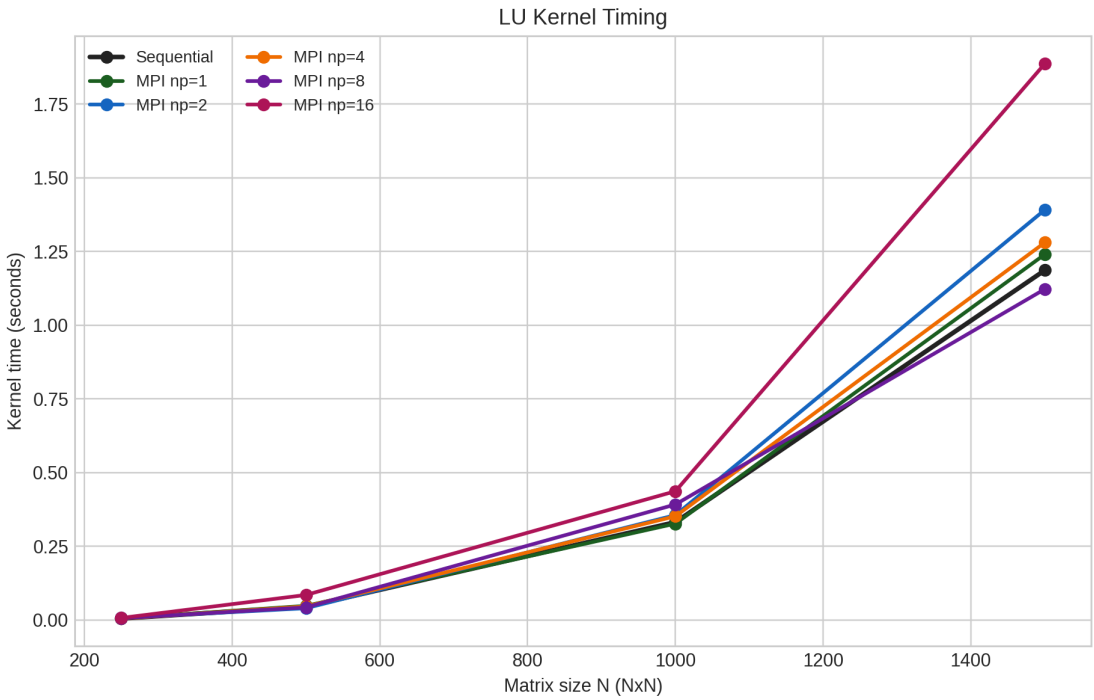

Figure 5: LU kernel timings across process counts.

5 Conclusion

This benchmark sweep confirms a clear split in behavior across the current implementations. Matrix multiplication, REF, and RREF all benefit from additional MPI ranks on the tested sizes, with RREF showing the strongest measured speedups. LU does not currently show comparable scaling because it is still executed only on rank 0 in the test harness. As a result, the new heap backed matrix storage removes the old size cap, but LU still needs a true distributed algorithm before higher MPI process counts will help materially.